Exercise: Reading the Enhanced Curriculum C Program for Engineering Comprehension

Purpose

This exercise guides you through a close reading of the **enhanced curriculum** C source code. The goal is not only to understand what the program does, but to evaluate its capabilities as a software-engineering artifact and as an engineering-design solution.

The program is an obfuscated-style C implementation using `xN` identifiers. Treat this as a reverse-engineering and design-comprehension task: infer roles, reconstruct intent, and judge the architecture from evidence in the code.

Learning outcomes

By completing this exercise, you should be able to:

1. Explain the program's observable capability in plain engineering language.
2. Map obfuscated identifiers to meaningful conceptual names.
3. Describe how the program generates fixed-width combinations from an input alphabet.
4. Explain why fixed-width integer types and overflow checks matter.
5. Identify the output abstraction and explain why it improves testability and portability.
6. Evaluate the program against software-engineering qualities: correctness, safety, modularity, maintainability, extensibility, and testability.
7. Propose engineering-design improvements while preserving the program's core behaviour.

Source under study

Use the enhanced curriculum C source code, headed as `configure.md` / `Curriculum`, containing:

- fixed-width integer use via `uint64_t`
- a digit alphabet
- a maximum output width constant
- an output sink structure with function pointers
- a safe exponentiation function
- a permutation/combination generator
- a file-backed output implementation
- a `main` function that generates width-7 digit strings into `x33.txt`

Part 1 — First-pass program comprehension

Read the whole source once without renaming anything.

Answer the following:

1. What file does the program write to?
2. What alphabet does it use?
3. What output width is selected in `main`?
4. How many output records should be produced for that width?
5. What is the shape of each output record?
6. What condition causes `main` to reject the selected width before opening the output file?
7. What does the program return when file opening fails?
8. What does the program return when generation fails after the file has been opened?

Expected reasoning

The answer should connect the constants, alphabet size, selected width, output loop, and newline emission. Do not simply say “it generates permutations”; describe the exact finite language produced by the program.

Part 2 — Identifier de-obfuscation table

Create a table that maps each identifier to its inferred purpose.

Identifier	Inferred name	Evidence from code	Engineering role
`x0`			
`x1`			
`x2`			
`x3`			
`x4`			
`x5`			
`x8`			
`x10`			
`x15`			
`x23`			
`x25`			
`x26`			
`x28`			
`x30`			
`x31`			
`x32`			

Challenge

After completing the table, rewrite the program's architecture in one paragraph using meaningful names only. Example style:

> The program defines a bounded alphabet and a maximum width, computes the number of combinations safely, enumerates each combination by interpreting the loop index in base `alphabet_size`, and writes each generated record through an output-sink abstraction backed by a file.

Part 3 — Capability model

Construct a capability model for the program.

3.1 Functional capabilities

Describe what the program can do.

Minimum points to cover:

- It can generate every fixed-width sequence over a fixed alphabet.
- It can write generated records to a file.
- It can detect arithmetic overflow before computing the total number of records.
- It can stop generation if output fails.
- It can reject invalid requested widths.

3.2 Non-capabilities

Describe what the program cannot currently do.

Consider:

- runtime selection of alphabet
- runtime selection of output width
- runtime selection of output file
- resumable generation
- progress reporting
- streaming to stdout
- configurable separator
- duplicate alphabet detection
- test harness or dependency injection beyond the sink interface

3.3 Engineering implication

For each non-capability, state whether it is:

- a deliberate simplification,
- a safety constraint,
- a maintainability issue,
- a usability limitation,
- or an extensibility opportunity.

Part 4 — Algorithmic comprehension

The core generator uses this pattern:

```
for each row index i:
    temp = i
    for each output position from right to left:
        char_index = temp % alphabet_size
        output[position] = alphabet[char_index]
        temp = temp / alphabet_size
```

Answer:

1. Why does the inner loop fill the output buffer from right to left?
2. What numeral system is being used to convert `i` into a sequence of alphabet indexes?
3. For alphabet `{0,1,2}` and width `2`, list the first six outputs in order.
4. What would change if the inner loop filled left to right without changing the arithmetic?
5. Why is the output buffer not null-terminated before being written?
6. Why is it safe to write exactly `width` bytes from the buffer?
7. Under what condition would this buffer-write approach become unsafe?

Part 5 — Arithmetic safety and overflow design

Study the safe power function.

```
result = 1
repeat exp times:
    if result > UINT64_MAX / base:
        fail
    result *= base
```

Answer:

1. What overflow is this guard preventing?
2. Why is division used before multiplication?
3. What precondition is silently assumed about `base`?
4. Does the current program satisfy that precondition?
5. What would happen if `base == 0`?
6. Should the safe power function itself check for `base == 0`? Defend your answer.
7. What is the largest valid width for alphabet size `10` in `uint64_t` arithmetic?
8. Why does the program still impose `x0 == 10` even though `uint64_t` could represent `10^19`?

Design note

Separate **numeric representability** from **buffer capacity** and **practical output volume**. A value can fit in `uint64_t` while still being impossible or unreasonable to generate in real time.

Part 6 — Output abstraction and design quality

The program defines a structure containing:

- a generic context pointer
- a function pointer for writing a buffer
- a function pointer for writing a character

Answer:

1. What design pattern does this resemble in C?
2. What concrete output implementation is provided?
3. How does this abstraction separate generation logic from file I/O?
4. How could you implement a memory-backed sink for unit tests?
5. How could you implement a counting sink that verifies the number of records without writing a file?
6. What failure path is enabled by returning `bool` from the sink functions?
7. What resource-management responsibility remains outside the abstraction?

Mini-design task

Sketch a `CountingSink` design in pseudocode. It should count:

- bytes written
- newline characters written
- number of records emitted

Do not write full C unless required by your instructor.

Part 7 — File I/O correctness

Inspect the file-backed sink.

Answer:

1. What mode is used to open the file?
2. What happens to any existing file with the same name?
3. How does the buffer-write function check success?
4. How does the character-write function check success?
5. Where is the file closed?
6. Is the file closed if generation fails?
7. Is the file closed if opening the file fails?
8. What risks remain around `fclose` errors?
9. Should `file_sink_close` return a status? Explain the trade-off.

Part 8 — Engineering-design review

Score the program from 1 to 5 in each category and justify each score with code evidence.

Category	Score	Evidence	Improvement
Correctness			
Arithmetic safety			
Memory safety			
I/O error handling			
Modularity			
Testability			
Maintainability			
Configurability			
Scalability			
Usability			

Guidance

A high score requires evidence, not optimism. A low score should identify a concrete engineering improvement.

Part 9 — Software-engineering capability statement

Write a concise capability statement in this format:

> This program is capable of [capability] under [constraints]. It achieves this through [architectural mechanisms]. Its strongest engineering features are [features]. Its primary limitations are [limitations]. It would be suitable for [use cases] but unsuitable for [use cases] unless [changes].

Your statement should mention:

- finite exhaustive generation
- bounded width
- safe arithmetic guard
- sink abstraction
- file output
- lack of runtime configurability
- impracticality of large output spaces

Part 10 — Engineering-design extension proposal

Propose one extension that improves the program without destroying its safety properties.

Choose one:

1. CLI-configurable width and output path

2. runtime-configurable alphabet
3. stdout sink
4. memory sink for testing
5. progress reporting
6. resumable generation
7. output-size preflight warning
8. duplicate-character detection in the alphabet

For your chosen extension, provide:

- user story
- new inputs
- validation rules
- affected functions
- new failure modes
- test cases
- safety argument

Part 11 — Test design

Design tests for the program.

Required test categories

1. **Unit tests for safe power**
    - small valid powers
    - exponent zero
    - boundary before overflow
    - overflow detection
    - base zero behaviour
2. **Generator tests using a fake sink**
    - width 1 over a small alphabet
    - width 2 over a small alphabet
    - sink write failure on record N
    - sink newline failure
    - verify generation stops after failure
3. **File sink tests**
    - invalid path
    - successful file creation
    - expected file size for a small configuration
    - close behaviour
4. **Main-level behaviour**
    - valid configured width

- zero width rejection
- width exceeding maximum rejection

Deliverable

Write a test matrix:

Test ID	Unit under test	Input/setup	Expected result	Engineering property verified
T1				
T2				
T3				

Part 12 — Risk analysis

Complete a risk table.

Risk	Cause	Impact	Existing mitigation	Proposed improvement
Very large output file				
Arithmetic overflow				
Buffer overrun				
File-open failure				
Mid-generation write failure				
Ambiguous obfuscated identifiers				
Inflexible hard-coded settings				
Silent close failure				

Part 13 — Refactoring task

Refactor the naming only. Do not change behaviour.

Produce a mapping and then rewrite the following as meaningful C names:

- constants
- alphabet array
- sink structure
- safe power function
- generator function
- file sink functions
- local variables in the generator
- `main` constants and sink variable

Constraints

- Preserve the same generated output.
- Preserve the same return-code behaviour.
- Preserve the same file target.
- Preserve the same arithmetic guard.
- Preserve the same maximum width.

Part 14 — Written reflection

Answer in 300–500 words:

> What does this program reveal about the relationship between low-level C programming, software-engineering discipline, and engineering design?

Your answer should discuss:

- explicit bounds
- abstraction through function pointers
- error propagation
- resource lifetime
- algorithmic scaling
- the difference between a working program and an engineered program

---

Part 15 — Mastery check

Answer without looking back at your notes.

1. What is the program's core algorithm?
2. What prevents integer overflow?
3. What prevents buffer overflow?
4. What abstraction separates generation from output?
5. What hard-coded choices limit the program's use?
6. What is the first extension you would implement and why?
7. What is the strongest evidence that the program was written with safety-critical design principles in mind?
8. What is the strongest evidence that the program is still only a small curriculum example rather than a production tool?

Optional advanced challenge

Compare this enhanced curriculum program with a more advanced configurable version.

Focus on how the design changes when these capabilities are added:

- CLI flags
- config-file parsing
- micro UI
- multiple flow types

- stdout output
- prefix, suffix, and separator handling
- multiple object specifications

Write a short comparison:

Design aspect	Enhanced curriculum program	Advanced configurable program	Engineering consequence
Input configuration			
Flow selection			
Output targeting			
Validation burden			
Test surface			
Maintainability			
User capability			

Submission checklist

Submit:

- completed identifier mapping table
- capability and non-capability model
- algorithm explanation
- safety analysis
- design-quality score table
- one extension proposal
- test matrix
- risk table
- 300–500 word reflection

Evaluation rubric

Criterion	Excellent	Satisfactory	Needs work
Program comprehension	Accurately explains exact output language and control flow	Explains general purpose but misses details	Vague or incorrect description
Identifier mapping	Most identifiers mapped with evidence	Some mappings correct	Mostly guessed
Safety reasoning	Clearly separates arithmetic, buffer, and I/O safety	Mentions safety but lacks detail	Treats all safety issues as the same

Criterion	Excellent	Satisfactory	Needs work
Architecture analysis	Explains sink abstraction and modularity	Identifies functions but not architecture	Only describes syntax
Engineering judgement	Balanced trade-off analysis	Some useful recommendations	Unsupported opinions
Test design	Covers unit, integration, failure, and boundary cases	Covers happy paths and some boundaries	Minimal or no failure testing
Extension proposal	Preserves safety and defines validation/failure modes	Plausible but incomplete	Feature idea without design
Reflection	Connects C details to engineering design principles	General reflection	Superficial summary

Instructor notes / answer anchors

Use these only after attempting the exercise.

- The program enumerates fixed-width strings over the digit alphabet `0`–`9`.
- With width `7`, it attempts to generate `10^7` records.
- Each record is seven characters followed by a newline.
- The output target is `x33.txt`.
- The generator interprets the loop counter as a base-10 number over the alphabet indexes.
- The sink abstraction is a C form of dependency inversion: the generator does not know whether output goes to a file, memory, stdout, or a test double.
- The overflow guard checks multiplication before performing it.
- The maximum width protects the fixed-size stack buffer.
- The program is more engineered than a quick script because it uses bounded buffers, explicit failure returns, fixed-width integers, and output abstraction.
- The program is not a production tool because key parameters are hard-coded, close errors are ignored, huge output is not preflighted, and there is no built-in test harness.